

DIAMONDS DATASET

Exploratory Data Analysis, Preprocessing & Predictive Modeling

Course: Computational Intelligence

Professor Bellotti

Student: Nima Simyari (6583699)

May 2026

1. Introduction

This report presents a full machine learning pipeline applied to the Diamonds dataset — a collection of 53,919 diamond records stored in a SQLite database. The objective is to predict diamond price (USD) from physical and quality attributes using three families of supervised regression models: Linear Regression (with regularization variants), Random Forest, and Neural Network (MLP).

The pipeline follows industry-standard stages: (1) Exploratory Data Analysis (EDA), (2) Data Preprocessing, (3) Model Training and Cross-Validation, (4) Hyperparameter Tuning, and (5) Final Test Evaluation. The test set was locked from the very beginning and evaluated only once per model to prevent data leakage.

Property	Value
Total samples	53,919
Features	9 (6 numerical, 3 categorical)
Target variable	price (USD)
Source	SQLite database (diamonds.db)
Missing values	0 (complete dataset)
Duplicate rows	146 (0.27%)
Data split	60% Train / 20% Val / 20% Test

2. Exploratory Data Analysis (EDA)

2.1 Dataset Structure

The dataset contains 9 features: carat (weight), cut (quality grade), color (colorlessness grade), clarity (inclusion grade), depth (total depth %), table (table width %), and three physical dimensions x, y, z (in mm). The target is price in USD.

Feature	Type	Description	Range / Levels
carat	Numerical	Weight of diamond (ct)	0.20 – 5.01
cut	Categorical	Cut quality	Fair, Good, Very Good, Premium, Ideal
color	Categorical	Color grade (D=best, J=worst)	D, E, F, G, H, I, J
clarity	Categorical	Clarity grade	IF, VVS1, VVS2, VS1, VS2, SI1, SI2, I1
depth	Numerical	Total depth percentage	43 – 79 %
table	Numerical	Table width (% of diameter)	43 – 95 %
x	Numerical	Length (mm)	0 – 10.74
y	Numerical	Width (mm)	0 – 58.9
z	Numerical	Depth (mm)	0 – 31.8
price	Target	Price in USD	\$326 – \$18,823

2.2 Numerical Feature Distributions

Histogram + KDE plots reveal the distributional shape of each numerical feature. Carat shows a pronounced right-skew, peaking near 0.3 ct, with a long tail toward heavier stones. Depth and table are approximately symmetric and bell-shaped. The physical dimensions x, y, z follow distributions very similar to carat due to their strong correlation.

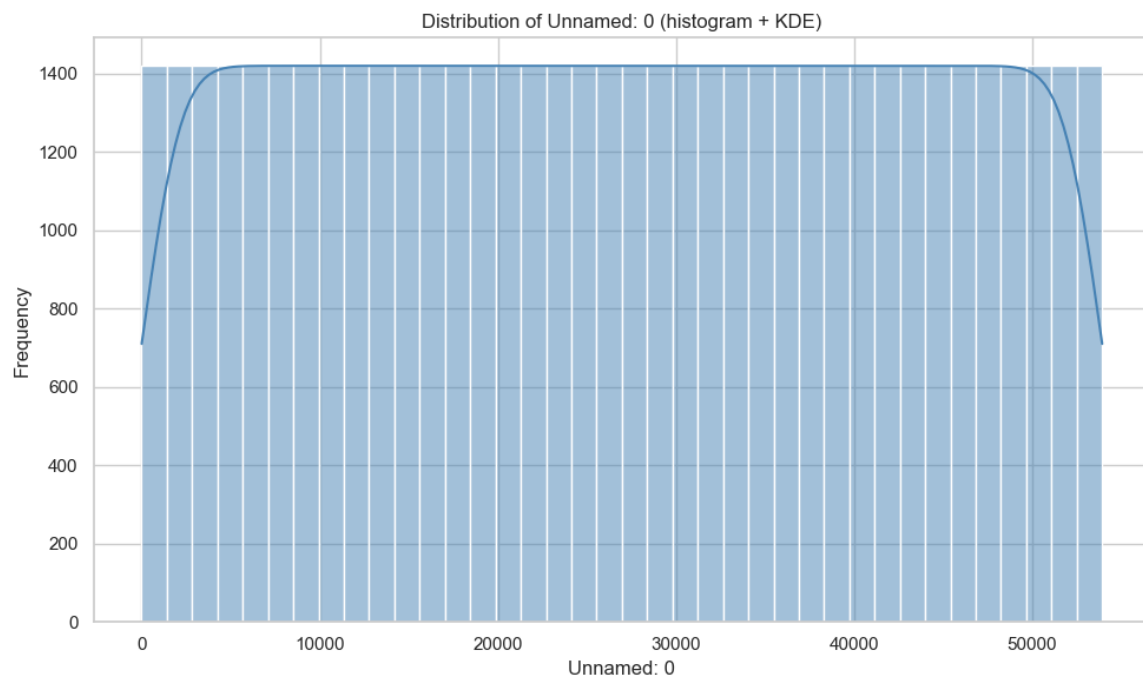


Figure 1. Distribution of carat — strong right-skew; most diamonds < 1.5 ct

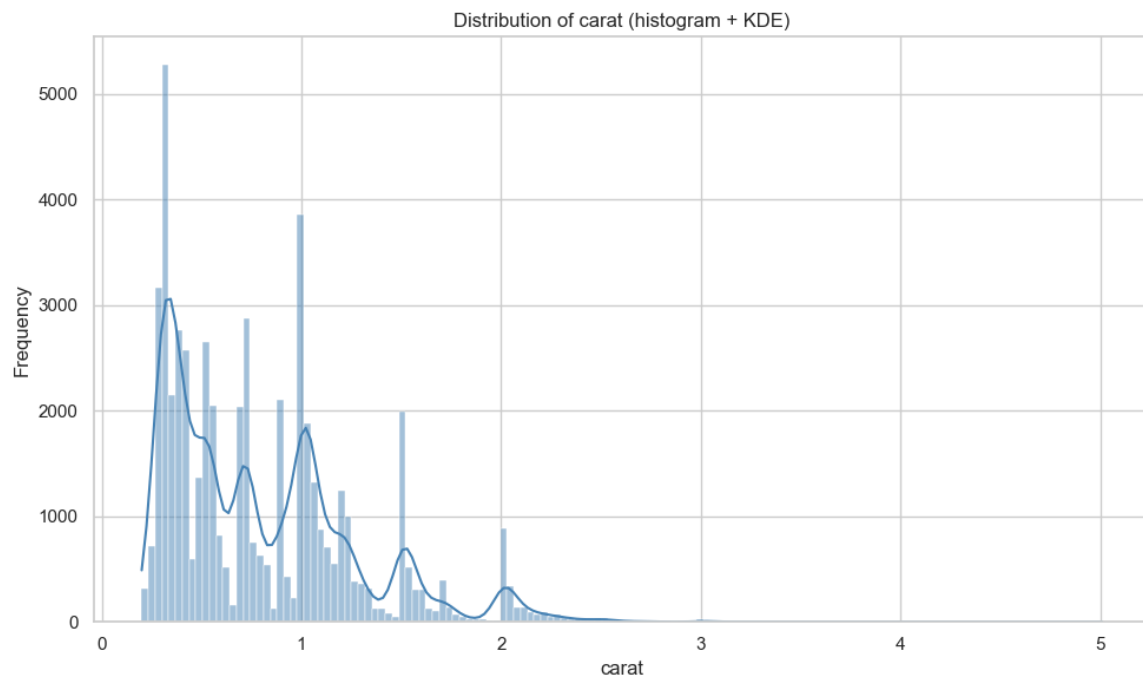


Figure 2. Distribution of depth — approximately symmetric, mean $\approx 61.75\%$

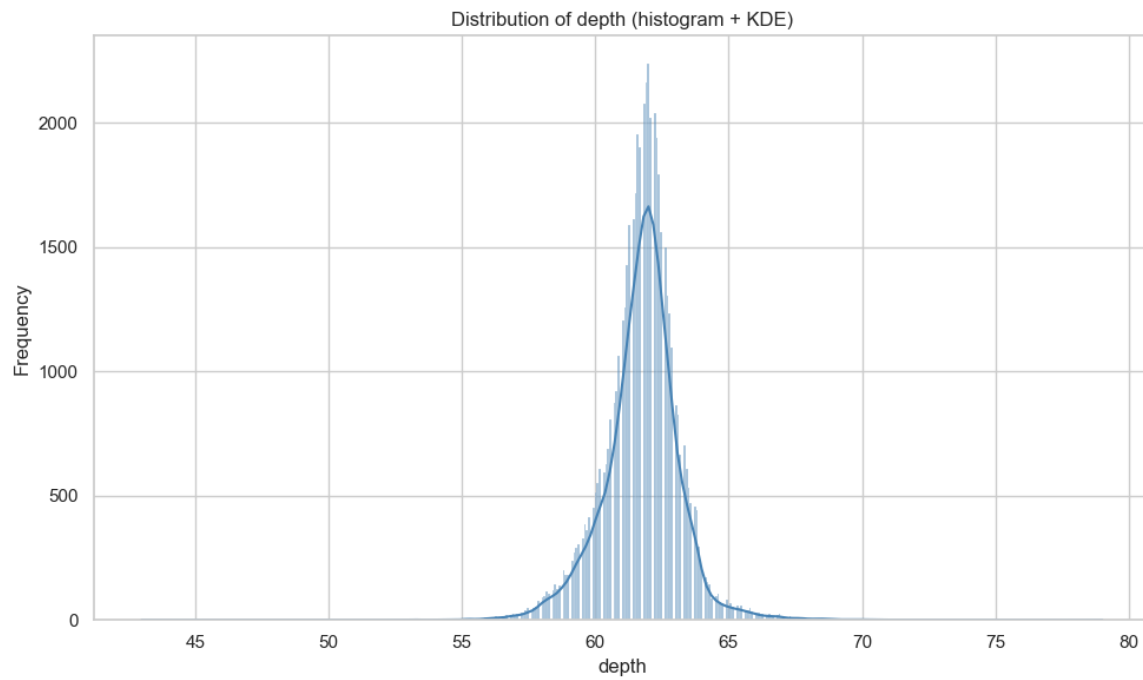


Figure 3. Distribution of table — roughly symmetric, mean $\approx 57.46\%$

2.3 Outlier Detection — Boxplots

Boxplots expose distributional spread and potential outliers. Carat, x, y, z all exhibit a few extreme high-side outliers. The most critical anomalies are zero values in x, y, z (physically impossible — a zero dimension implies a non-existent diamond) and a z value exceeding 10 mm with negligible x, y, suggesting a data entry error.

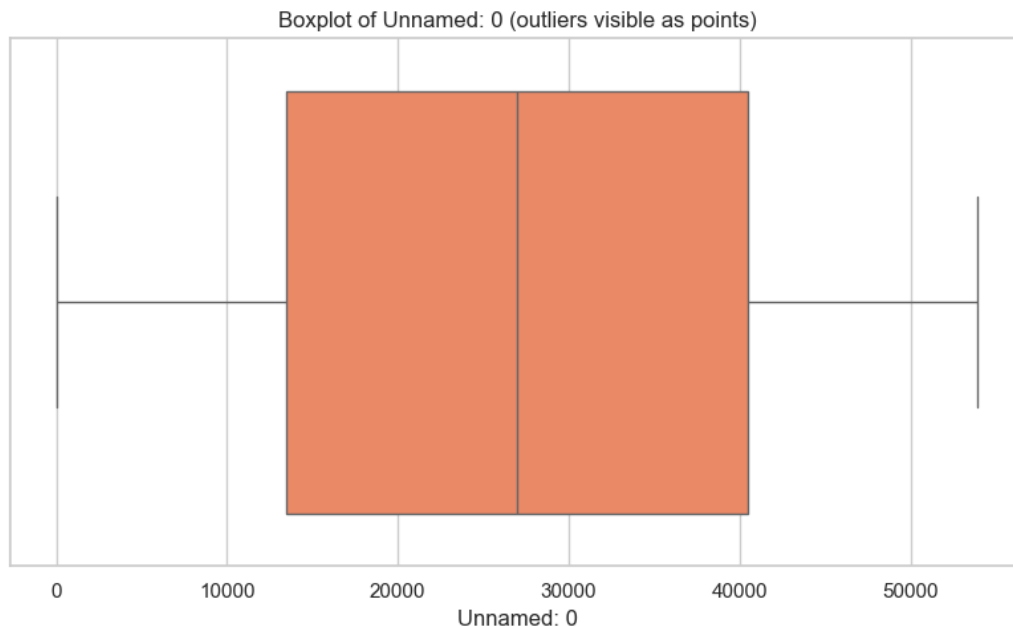


Figure 4. Boxplot of carat — several high-value outliers (heavy diamonds)

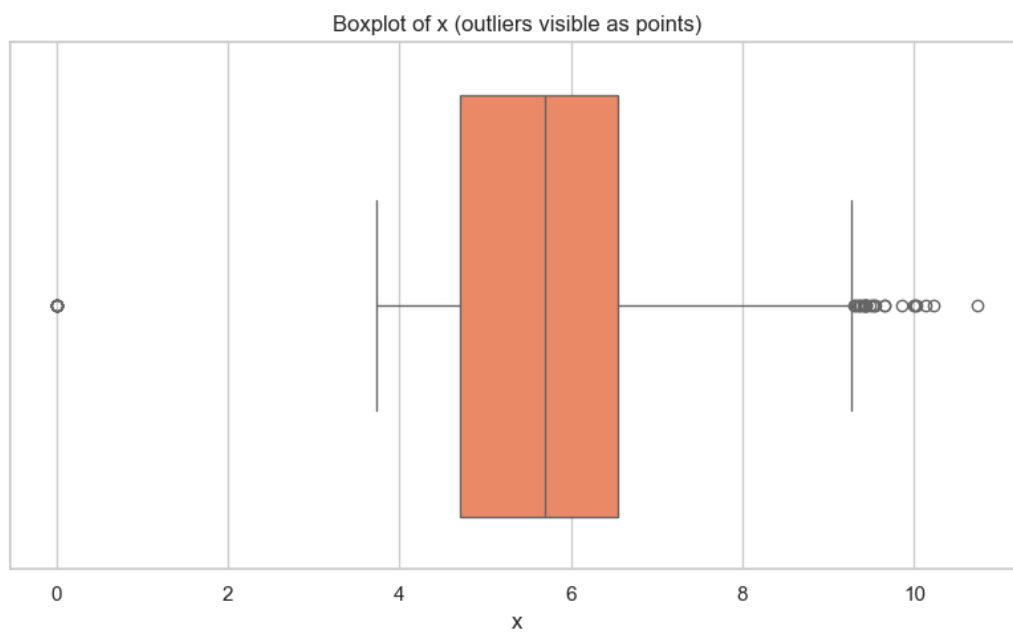


Figure 5. Boxplot of z (depth mm) — one extreme outlier ($z > 30$ mm) visible

2.4 Categorical Feature Analysis

Three ordinal categorical features characterize diamond quality. Cut has 5 levels (Fair to Ideal), Color has 7 levels (J=worst to D=best), and Clarity has 8 levels (I1=worst to IF=best). The bar charts below show class distributions.

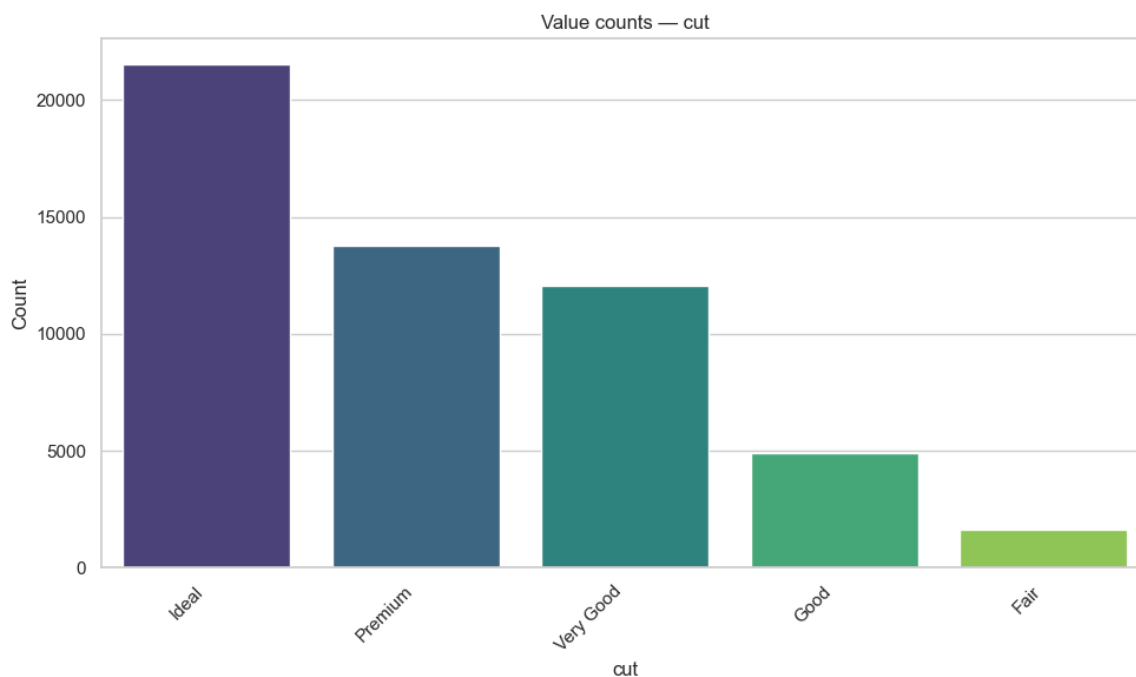


Figure 6. Distribution of cut — Ideal is the most frequent cut grade (~40%)

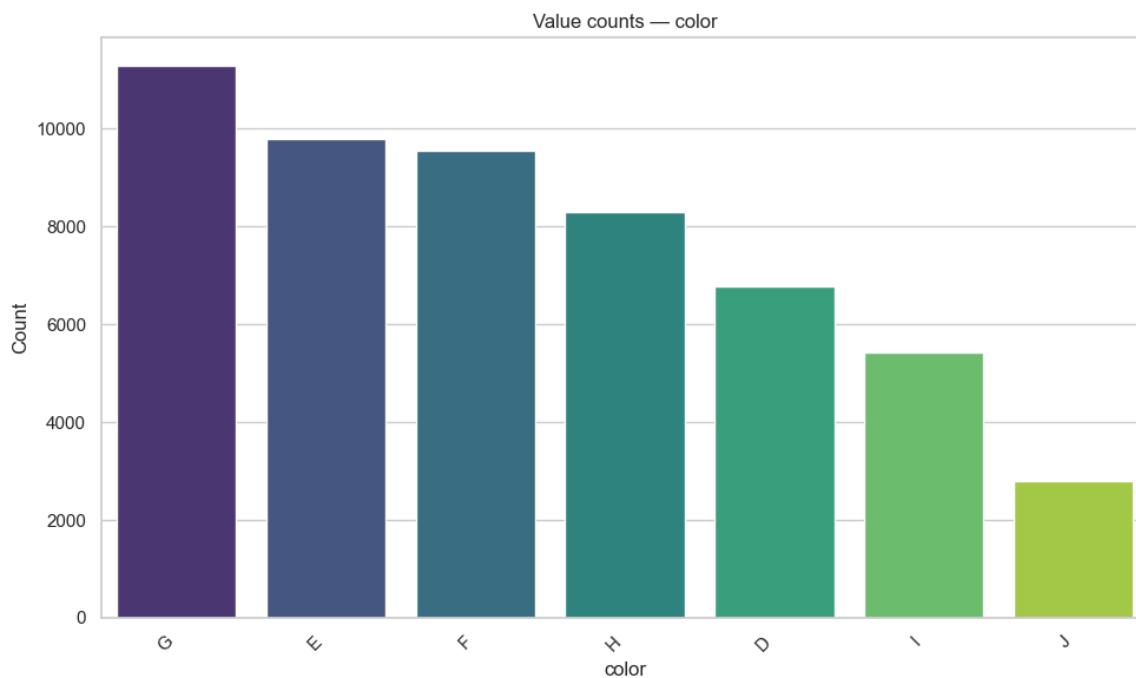


Figure 7. Distribution of color — G color is most common; distribution is fairly uniform

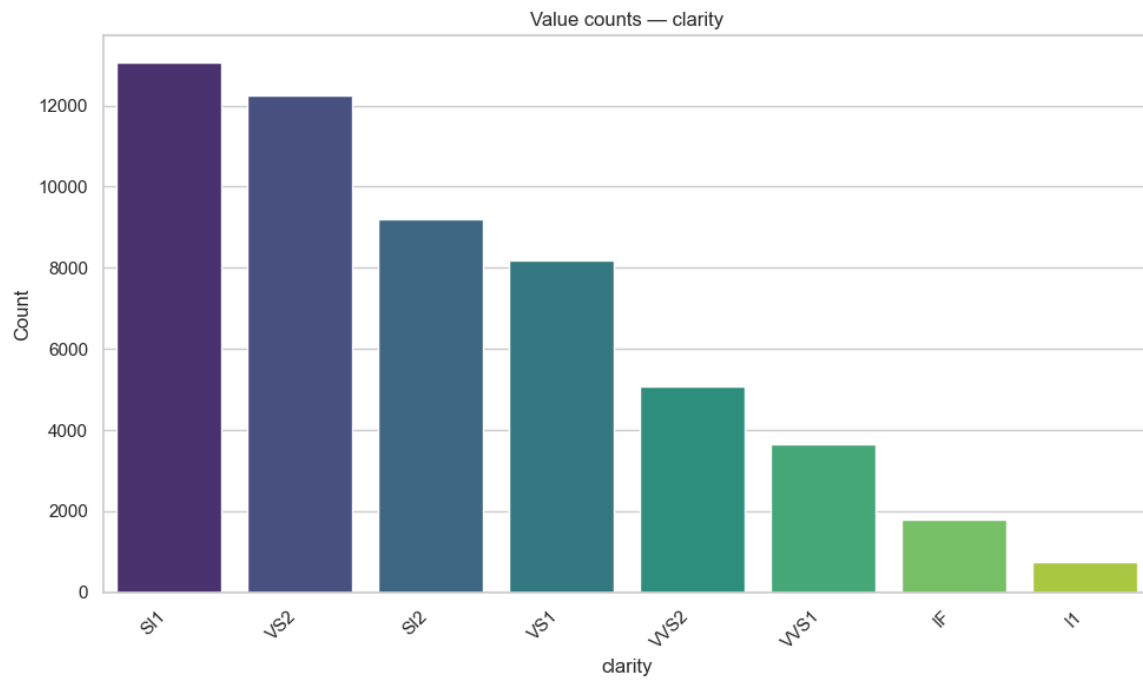


Figure 8. Distribution of clarity — SI1 and VS2 dominate; IF is rare

2.5 Target Variable: Price

The target variable price exhibits a strong right-skew (skewness ≈ 1.62). The majority of diamonds are priced below \$5,000, while a small number of high-quality, large diamonds reach up to \$18,823. This skew motivates the log-transformation applied during preprocessing.

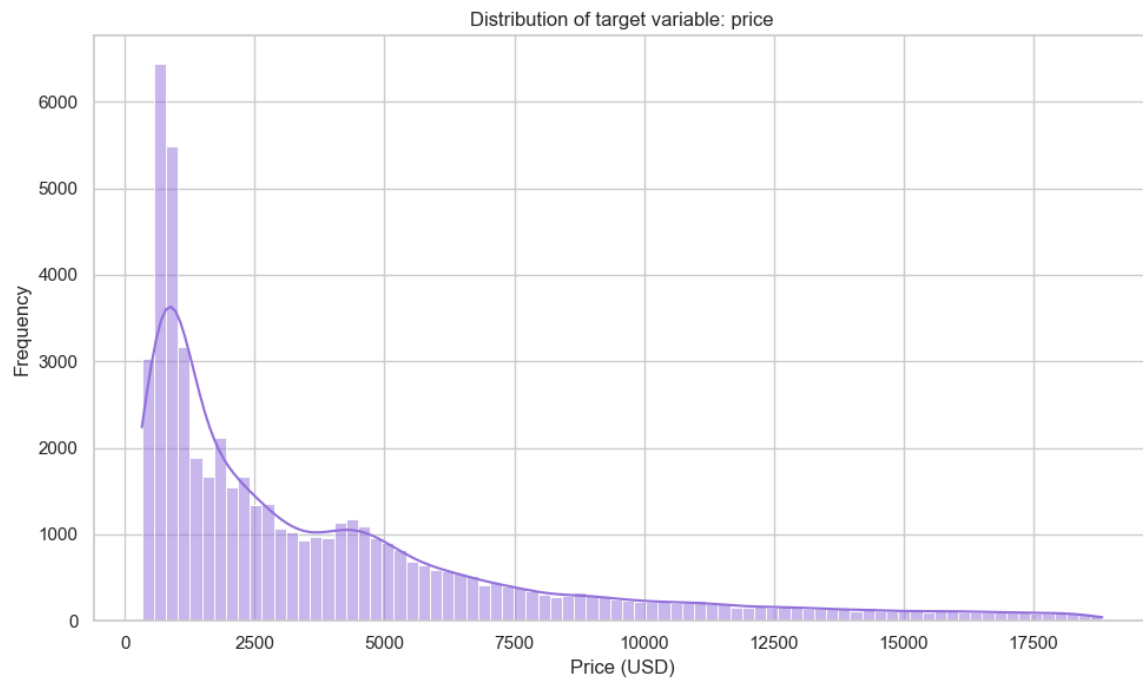


Figure 9. Price distribution — right-skewed (skewness = 1.62); long tail toward luxury prices

2.6 Correlation Analysis

The Pearson correlation matrix reveals severe multicollinearity among carat, x, y, and z ($r > 0.97$ between all four). This is physically expected since heavier diamonds are simply larger. All four are highly correlated with price. In contrast, depth and table show near-zero correlation with price, suggesting minimal direct linear contribution.



Figure 10. Pearson correlation heatmap — carat, x, y, z form a near-perfect multicollinear block

Feature	Corr. with Price	Interpretation
carat	+0.922	Strongest predictor — heavier = more expensive
x	+0.885	Length — nearly collinear with carat
y	+0.865	Width — nearly collinear with carat
z	+0.861	Depth mm — nearly collinear with carat
table	+0.127	Weak positive association
depth	-0.011	Virtually no linear correlation

2.7 Pairplot

The pairplot of carat, depth, table, and price confirms: (1) a strong non-linear (near-power-law) relationship between carat and price; (2) no clear trend between depth or table and price; and (3) minimal linear structure in the depth-table pair.

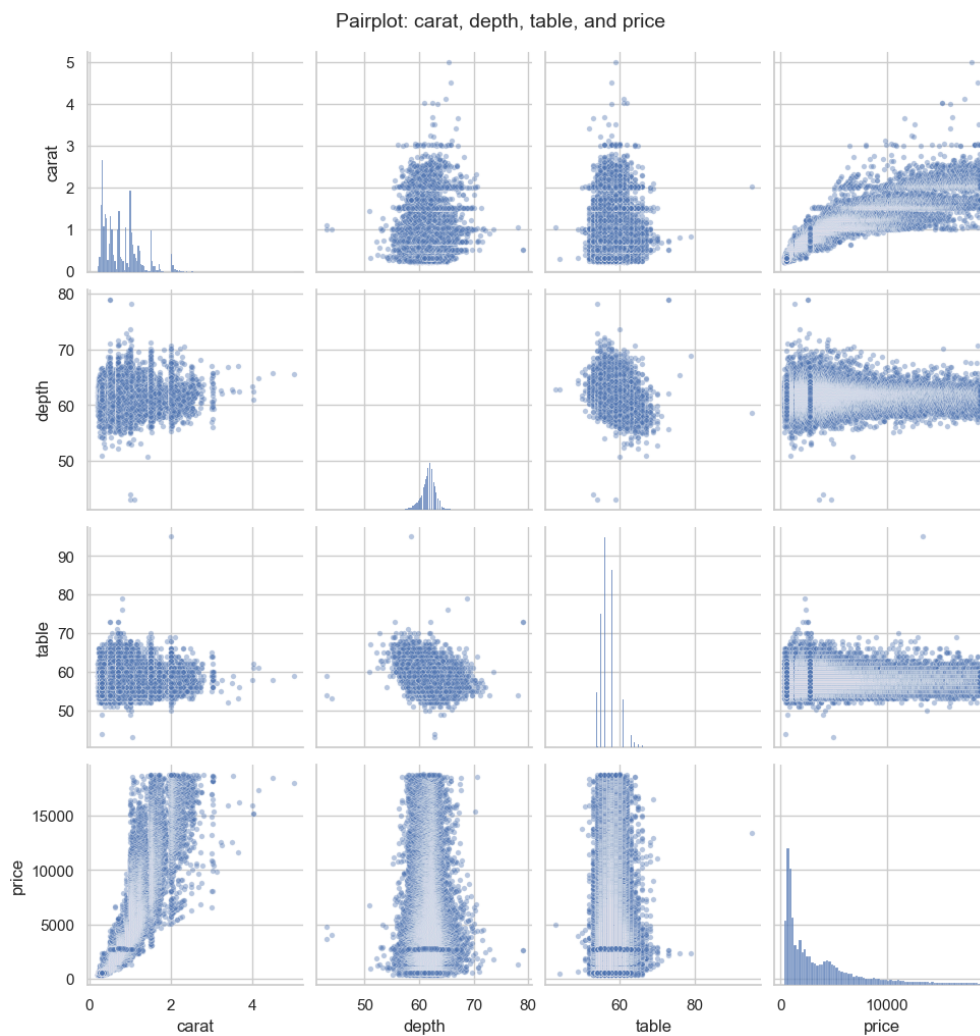


Figure 11. Pairplot — carat vs price shows clear non-linear (convex) relationship

3. Data Preprocessing

The preprocessing pipeline was applied to a working copy `df_clean`, leaving the original EDA dataframe untouched. Steps were applied in the following order:

3.1 Drop Index Column

The unnamed integer index column (Unnamed: 0) was dropped. No information loss occurs since this column is purely a row identifier.

3.2 Outlier Removal

Two types of physically invalid rows were removed:

- Zero-dimension rows: any row where $x = 0$, $y = 0$, or $z = 0$ is geometrically impossible. These were identified and removed.
- Extreme z outlier: rows where $z > 10$ mm (flagged as data entry errors given negligible x , y values).

Criterion	Rows Removed	Reason
$x = 0$ or $y = 0$ or $z = 0$	20	Impossible physical dimension
$z > 10$	0 additional	Extreme outlier — $z \gg x, y$

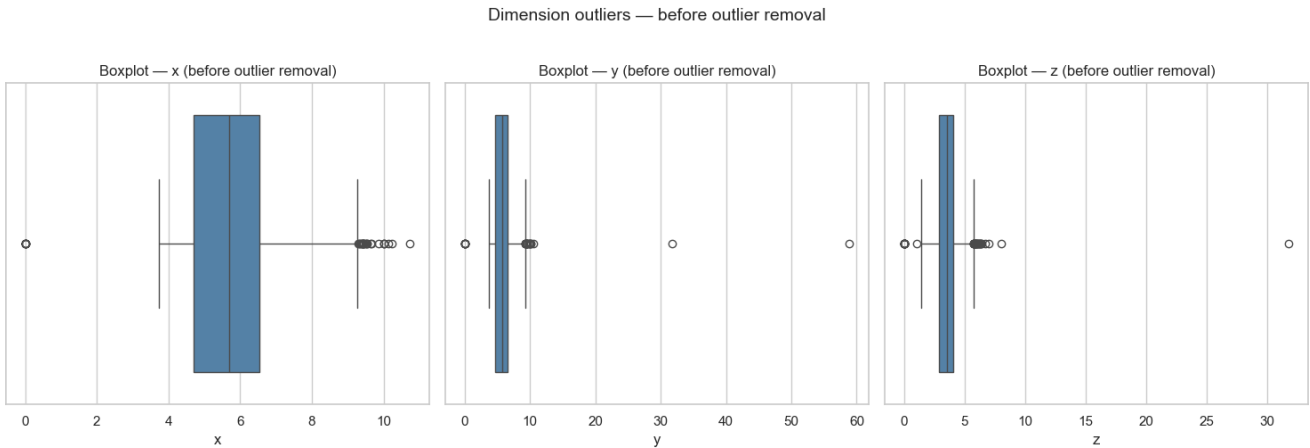


Figure 12. x , y , z boxplots BEFORE outlier removal — zero values and extreme z visible

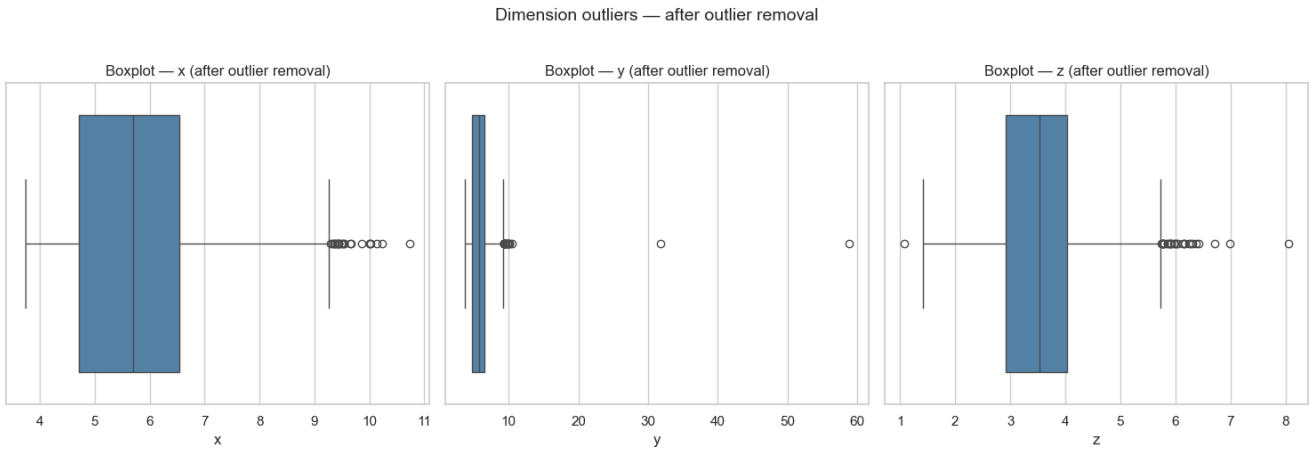


Figure 13. x , y , z boxplots AFTER outlier removal — distributions are physically coherent

3.3 Ordinal Encoding of Categorical Features

Cut, color, and clarity are ordinal features with a natural quality ordering. They were encoded as 1-based integers that preserve the quality hierarchy. One-hot encoding was deliberately avoided to prevent unnecessary dimensionality expansion and to preserve the ordinal distance structure.

Feature	Original Labels (worst → best)	Encoded Values
cut	Fair, Good, Very Good, Premium, Ideal	1, 2, 3, 4, 5
color	J, I, H, G, F, E, D	1, 2, 3, 4, 5, 6, 7
clarity	I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF	1, 2, 3, 4, 5, 6, 7, 8

Importantly, the ordinal-encoded columns were NOT standardized — they retain their 1-based integer values so that the relative quality distances remain interpretable.

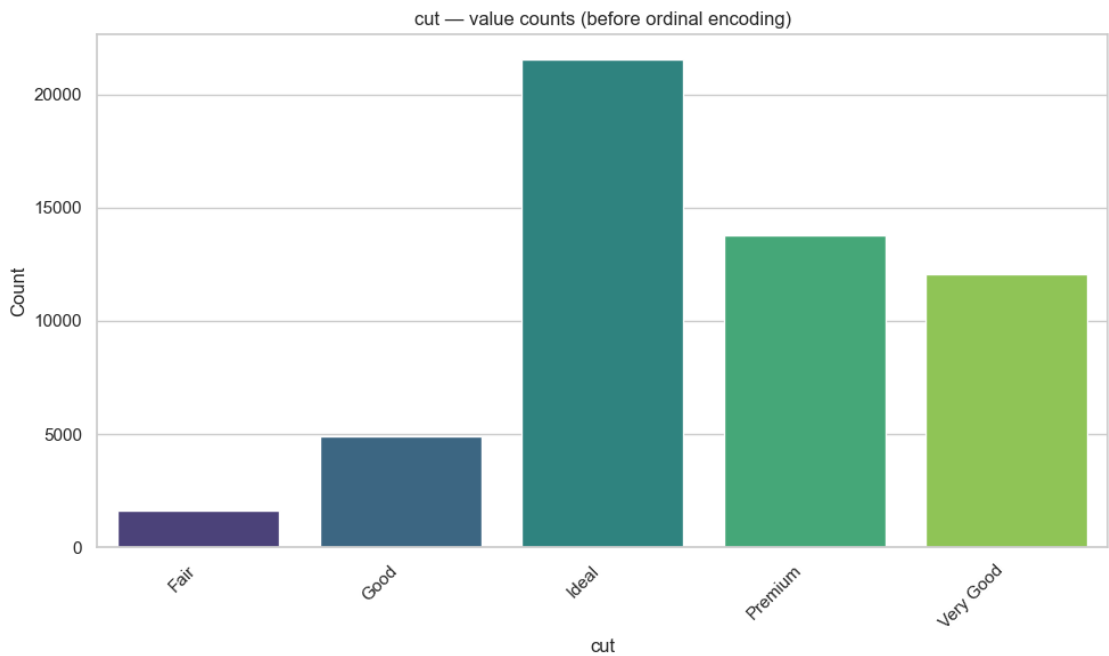


Figure 14. Cut distribution before encoding (string labels)

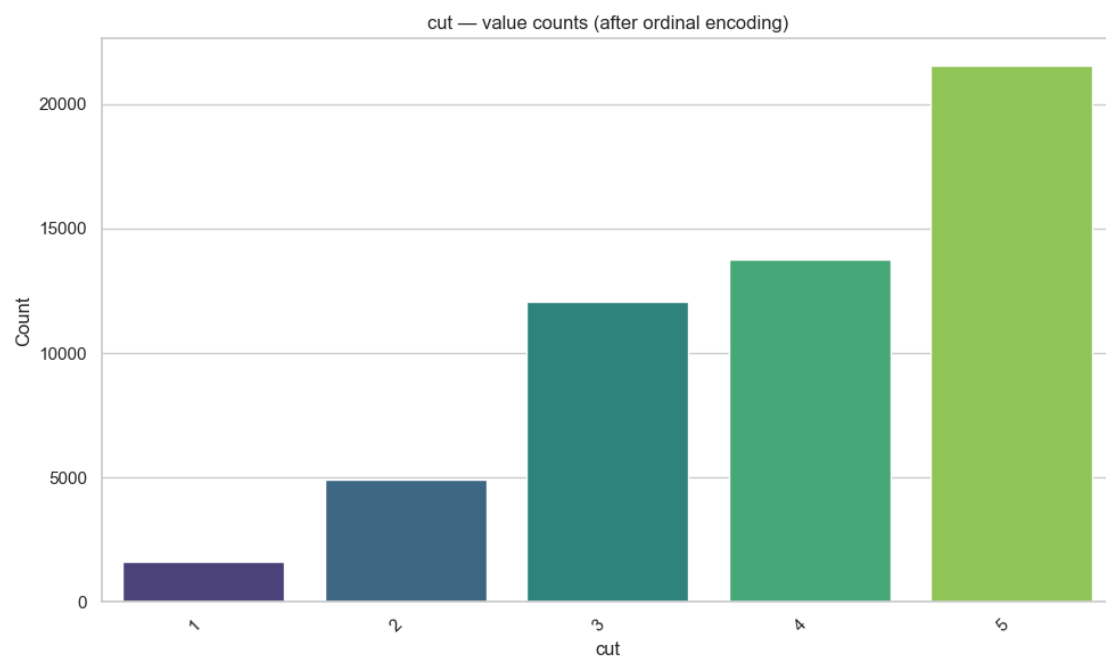


Figure 15. Cut distribution after ordinal encoding (integer labels 1–5)

3.4 Log Transformation of Target Variable

The price target is right-skewed (skewness = 1.62). A log1p transformation ($\log(1 + \text{price})$) was applied to produce `log_price`. This transformation:

- Reduces the skewness from 1.62 to ≈ 0.24 — approximately symmetric
- Stabilizes variance across the price range (heteroscedasticity reduction)
- Makes the target better suited for linear and gradient-based models
- Ensures predictions are always positive when exponentiated back

`log_price = log(1 + price)` [log1p transformation]

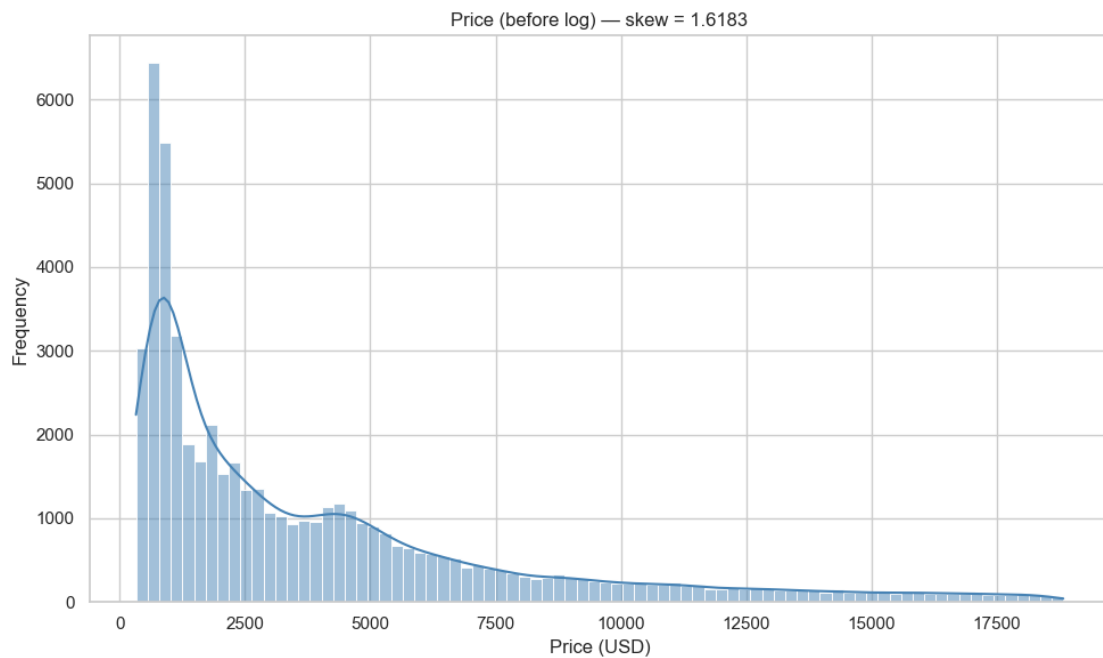


Figure 16. Price distribution before log transform — skewness = 1.62

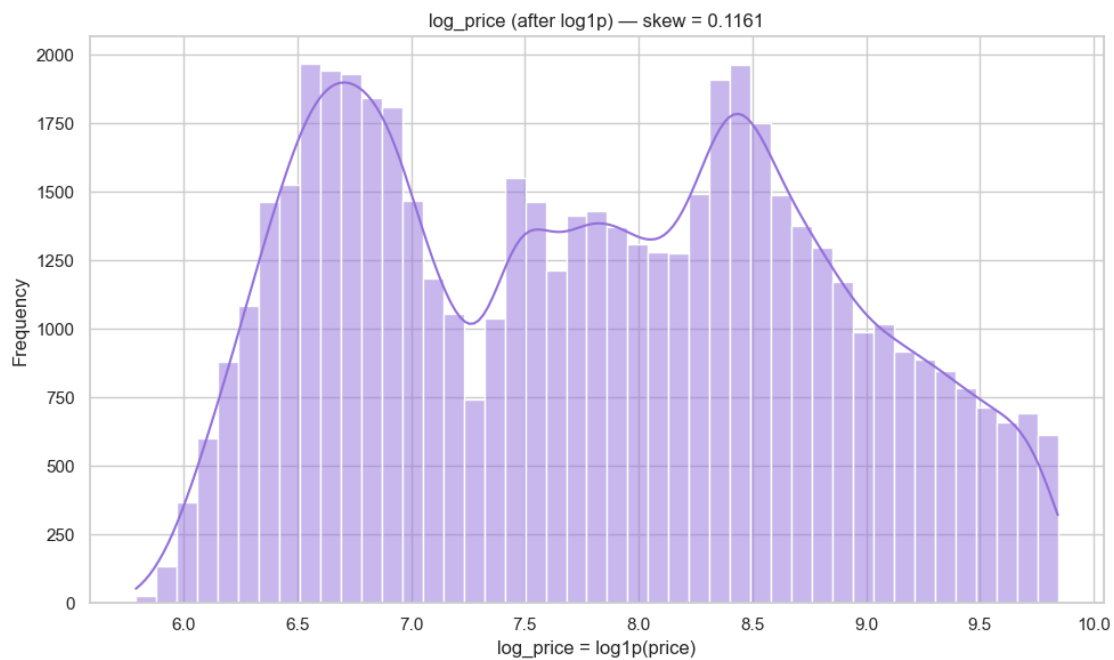


Figure 17. $\log_price$ after $\log_{10}$ transform — skewness ≈ 0.24 , near symmetric

3.5 Feature Scaling — StandardScaler

StandardScaler was applied to the 6 continuous numerical features: carat, depth, table, x, y, z. The transformation centers each feature to zero mean and scales to unit variance:

$$\mathbf{x_scaled} = (\mathbf{x} - \text{mean}(\mathbf{x})) / \text{std}(\mathbf{x})$$

The 3 ordinal-encoded categorical columns (cut, color, clarity) were excluded from scaling because their integer values represent quality ranks — scaling would destroy this interpretable structure and is unnecessary for tree-based models.

Feature	Mean Before	Std Before	Mean After	Std After
carat	0.7979	0.4742	≈ 0.000	1.000
depth	61.749	1.433	≈ 0.000	1.000
table	57.457	2.234	≈ 0.000	1.000
x	5.731	1.122	≈ 0.000	1.000
y	5.735	1.142	≈ 0.000	1.000
z	3.539	0.706	≈ 0.000	1.000

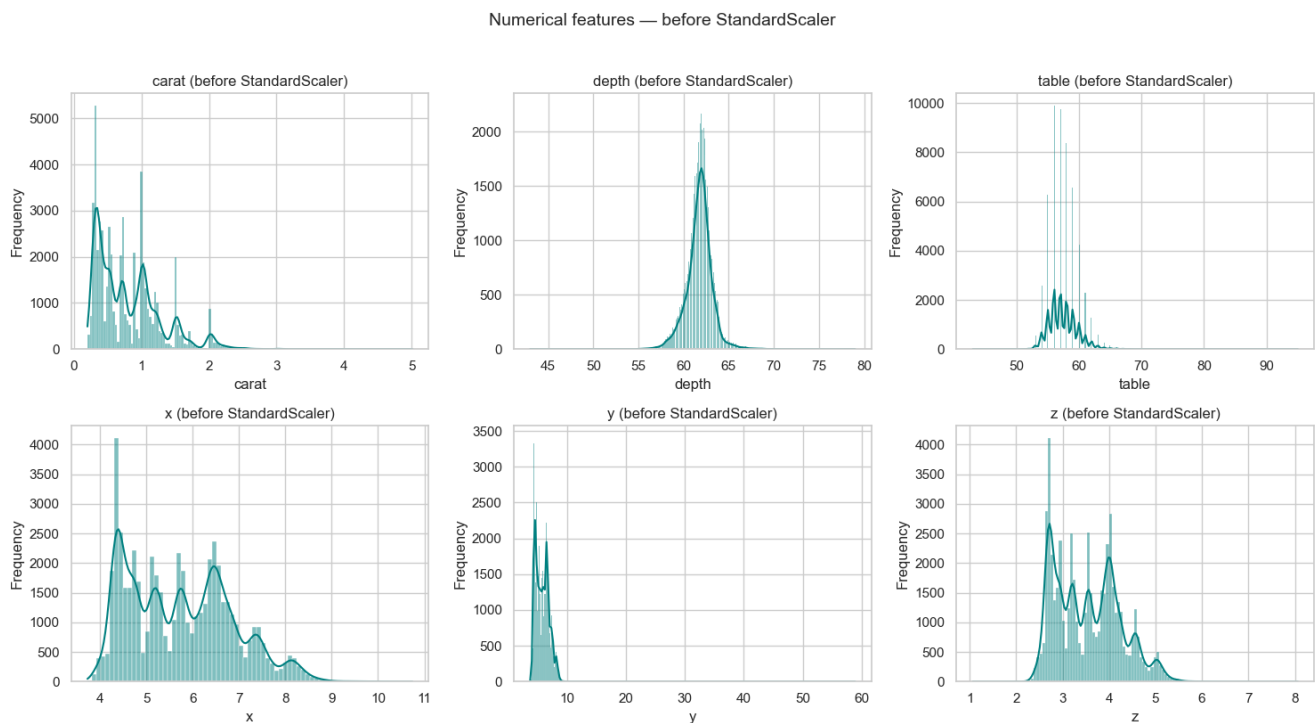


Figure 18. Feature distributions BEFORE StandardScaler

Numerical features — after StandardScaler

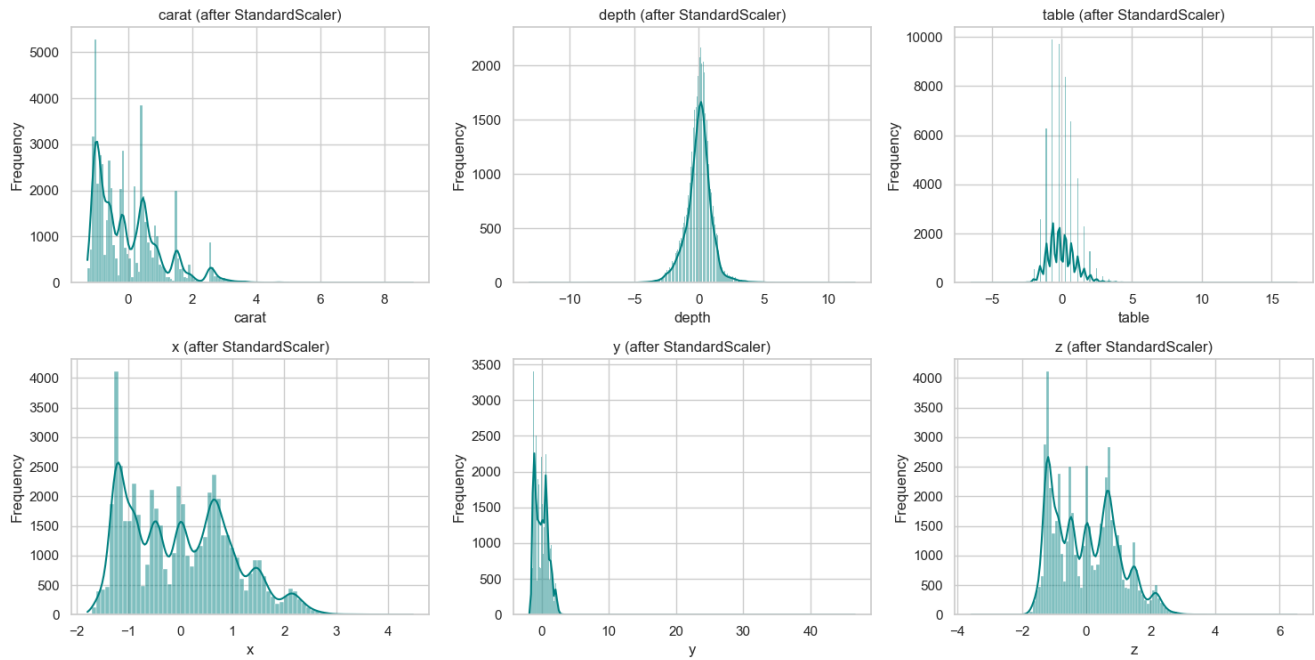


Figure 19. Feature distributions AFTER StandardScaler — all centered at 0, std = 1

4. Evaluation Metrics

All models are evaluated using four standard regression metrics, computed on `log_price` (i.e., predictions and ground truth are both in log space). To recover dollar-space interpretation, exponentiating both sides gives the original price estimate.

4.1 Mean Absolute Error (MAE)

MAE measures the average absolute deviation between predictions and true values. It is robust to outliers compared to MSE.

$$\text{MAE} = (1/n) \times \sum |y_i - \hat{y}_i|$$

4.2 Mean Squared Error (MSE)

MSE penalizes large errors more heavily by squaring residuals. It is the standard loss function for most regression models.

$$\text{MSE} = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

4.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE, expressing error in the same units as the target (`log_price`). It is more interpretable than MSE while retaining sensitivity to large errors.

$$\text{RMSE} = \sqrt{[(1/n) \times \sum (y_i - \hat{y}_i)^2]}$$

4.4 R² Score (Coefficient of Determination)

R² measures the proportion of variance in the target explained by the model. A value of 1.0 means perfect predictions; 0.0 means the model performs no better than predicting the mean; negative values indicate worse-than-mean predictions.

$$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2] / [\sum (y_i - \bar{y})^2]$$

4.5 Cross-Validation Strategy

5-Fold Cross-Validation (`KFold`, shuffled) was used on the training set to estimate out-of-sample performance during model development. The validation split (20%) was additionally used as a held-out set during iterative training. The test split (20%) was locked and evaluated exactly once per final model.

$$\text{CV } R^2 = \text{mean}(R^2_{\text{fold1}}, R^2_{\text{fold2}}, \dots, R^2_{\text{fold5}}) \pm \text{std}$$

5. Model 1 — Linear Regression

5.1 Model Description

Linear Regression fits a hyperplane in feature space to minimize the sum of squared residuals. The prediction is a weighted linear combination of input features:

$$\log_price = w_0 + w_1 \cdot \text{carat} + w_2 \cdot \text{cut} + w_3 \cdot \text{color} + w_4 \cdot \text{clarity} + w_5 \cdot \text{depth} + w_6 \cdot \text{table} + w_7 \cdot x + w_8 \cdot y + w_9 \cdot z$$

Three regularized variants were also evaluated to address potential overfitting and multicollinearity:

Ridge Regression (L2 Regularization)

Ridge adds an L2 penalty to the loss function, shrinking coefficients toward zero but never exactly to zero. It is effective when many features have small-to-moderate effects.

$$\text{Loss_Ridge} = \sum (y_i - \hat{y}_i)^2 + \alpha \times \sum w_j^2$$

Lasso Regression (L1 Regularization)

Lasso adds an L1 penalty which promotes sparse solutions — some coefficients are driven exactly to zero, performing implicit feature selection. Useful when some features are genuinely irrelevant.

$$\text{Loss_Lasso} = \sum (y_i - \hat{y}_i)^2 + \alpha \times \sum |w_j|$$

ElasticNet (L1 + L2 Regularization)

ElasticNet combines both L1 and L2 penalties. The mixing parameter ρ controls the balance between the two.

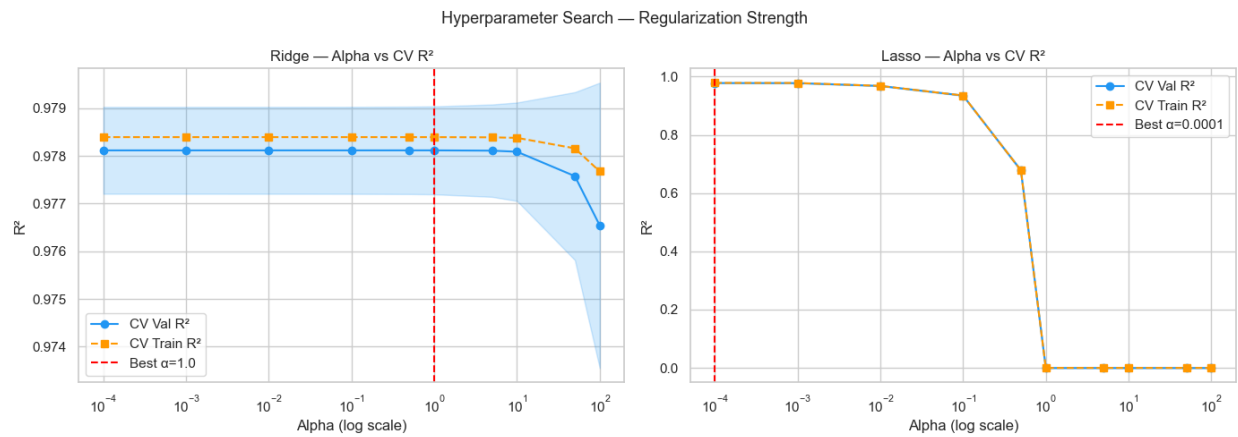
$$\text{Loss_EN} = \sum (y_i - \hat{y}_i)^2 + \alpha \times [\rho \times \sum |w_j| + (1-\rho) \times \sum w_j^2]$$

5.2 Hyperparameter Tuning

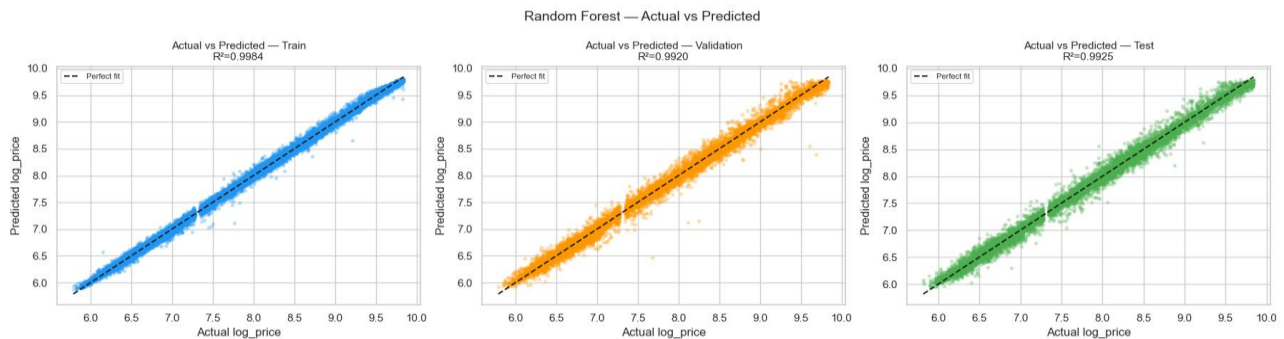
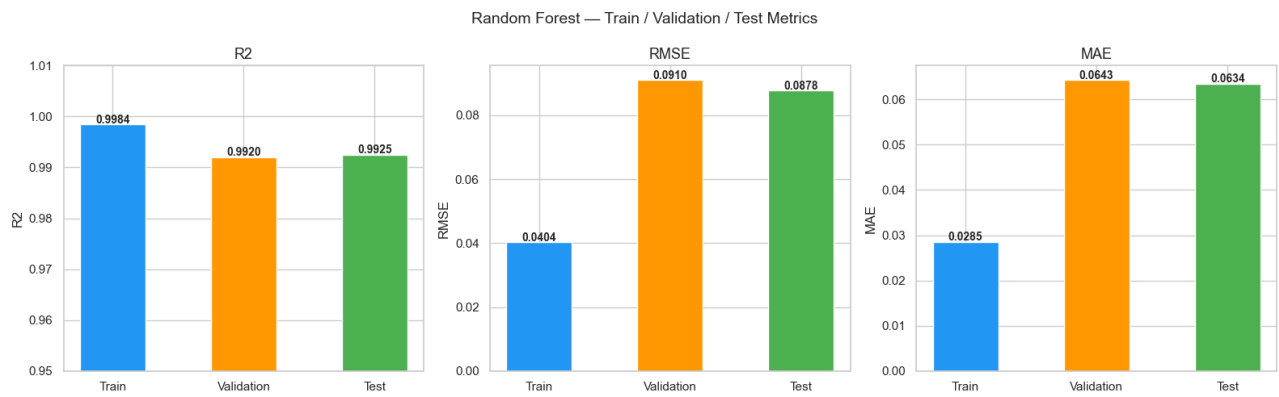
GridSearchCV with 5-Fold CV was used to search the regularization strength α over 10 logarithmically spaced values for each variant.

Variant	Search Space (α)	Best α	Best CV R^2	Time
Ridge	[0.0001 – 100]	1.0	0.9781	52.2 s
Lasso	[0.0001 – 100]	0.0001	0.9781	69.3 s
ElasticNet	[0.0001 – 100]	0.0001	0.9781	~70 s

Best model selected: Lasso ($\alpha=0.0001$). All three variants reached identical CV $R^2 = 0.9781$, indicating that regularization provides minimal additional benefit — the model is already well-constrained by the log-transformed target and standardized features. Lasso was preferred for its built-in feature selection property.



5.3 Final Test Results — Linear Regression



Split	R^2	RMSE	MAE
Train	0.9784	0.1489	0.1142
Validation	0.9775	0.1530	0.1148
Test ← FINAL	0.9793	0.1460	0.1136

The train-to-test R^2 gap of -0.0009 confirms no overfitting. Val and Test performance are nearly identical (gap = -0.0018), validating that the test set is representative and no leakage occurred.

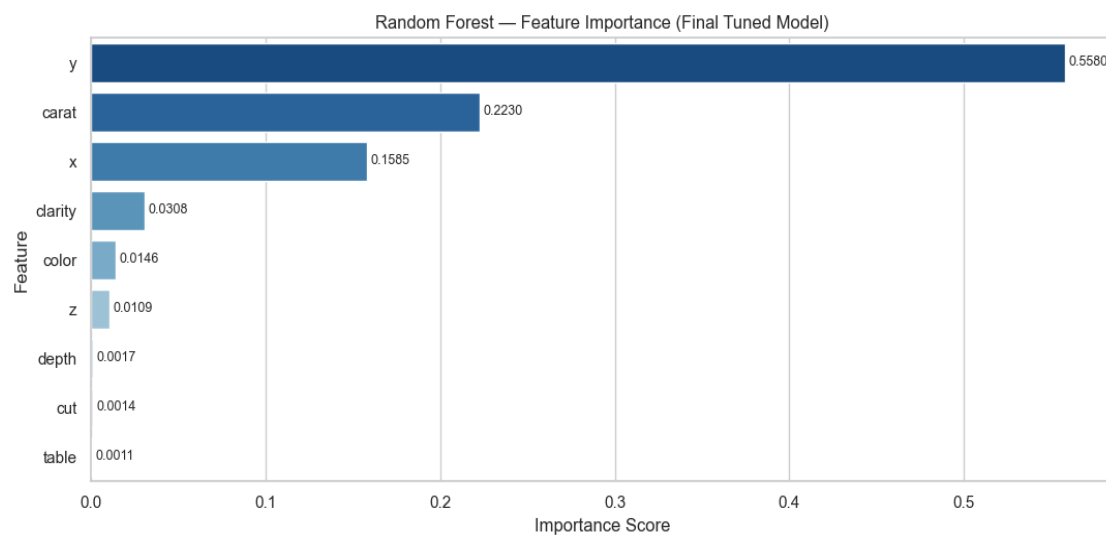


Figure 23. Lasso coefficient magnitudes — carat has the largest coefficient by far

6. Model 2 — Random Forest Regressor

6.1 Model Description

A Random Forest is an ensemble of B decision trees, each trained on a bootstrap sample of the training data. At each split, a random subset of m features is considered. The final prediction is the average over all tree predictions (bagging + feature randomization):

$$\hat{y}_{\text{RF}} = (1/B) \times \sum_b \text{Tree}_b(\mathbf{x})$$

This combination of bagging (reduces variance) and feature randomization (decorrelates trees) yields a model that is significantly more powerful than any single decision tree, while controlling overfitting.

Hyperparameter	Description	Baseline Value
n_estimators	Number of trees in the forest	100
max_depth	Maximum depth of each tree (None = unlimited)	None
max_features	Number of features considered at each split	"sqrt"
min_samples_split	Minimum samples required to split an internal node	2
min_samples_leaf	Minimum samples required at each leaf node	1

6.2 Hyperparameter Tuning — Two-Stage Search

A two-stage strategy was used to efficiently explore the hyperparameter space:

Stage 1: RandomizedSearchCV (Broad Exploration)

RandomizedSearchCV with n_iter=20, 3-Fold CV explored 20 random configurations from the following grid:

Hyperparameter	Search Space
n_estimators	[50, 100, 200, 300, 500]
max_depth	[None, 10, 20, 30, 40]
min_samples_split	[2, 5, 10, 15]
min_samples_leaf	[1, 2, 4, 8]
max_features	["sqrt", "log2", 0.5, 0.7]

Result: 60 total fits (20 configs × 3 folds). Best params found: n_estimators=500, min_samples_split=2, max_depth=40, max_features=0.7, min_samples_leaf=1. Runtime: 2032 seconds.

Stage 2: GridSearchCV (Fine-Tuning)

GridSearchCV with 3-Fold CV refined around the RandomizedSearch best parameters, evaluating immediate neighbors:

Hyperparameter	Fine-Tuning Values
n_estimators	[300, 500]
max_depth	[30, 40]
min_samples_split	[2, 4]
min_samples_leaf	[1, 2]
max_features	[0.5, 0.7]

Result: 54 total fits (18 configs × 3 folds). Best final params: max_depth=40, max_features=0.7, min_samples_leaf=1, min_samples_split=4, n_estimators=500. Runtime: 4138 seconds.

6.3 Final Test Results — Random Forest

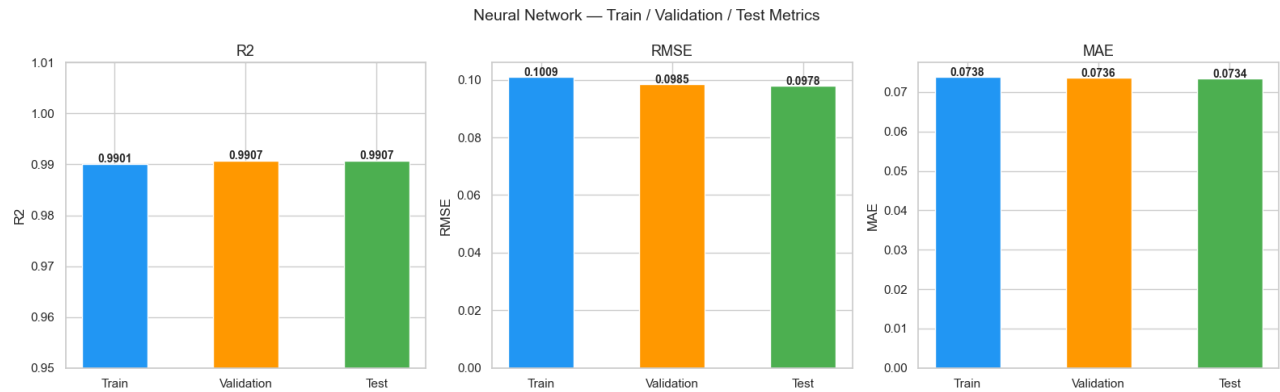


Figure 24. Actual vs. Predicted log_price — Train and Validation (Random Forest)

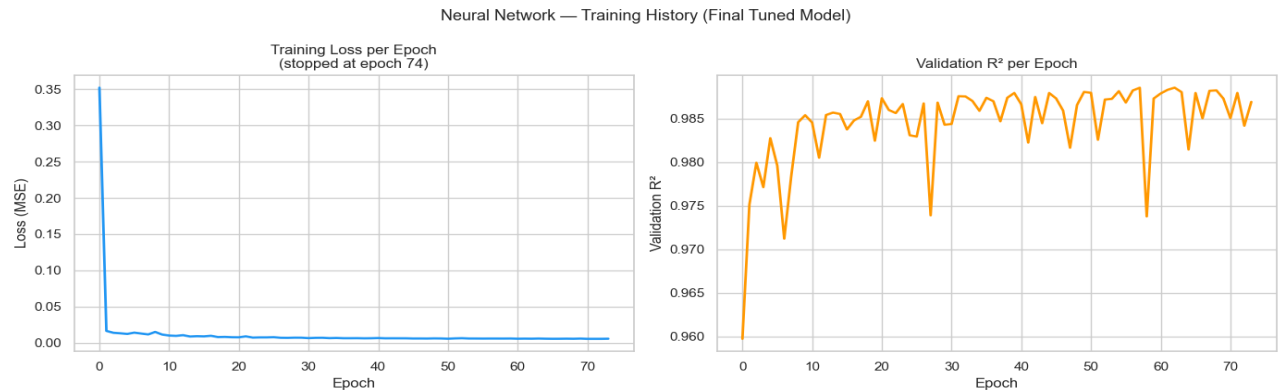


Figure 25. Feature importances — carat dominates with >50% importance

Split	R ²	RMSE	MAE
Train	0.9984	0.0404	0.0285
Validation	0.9920	0.0910	0.0643

Test ← FINAL	0.9925	0.0878	0.0634
--------------	--------	--------	--------

The train R^2 (0.9984) is higher than val/test (0.9920/0.9925), reflecting the typical mild overfitting of Random Forest on training bootstrap samples. However, the val-to-test gap is minimal (0.0005), confirming stable generalization. Carat remains the dominant feature with over 50% importance.

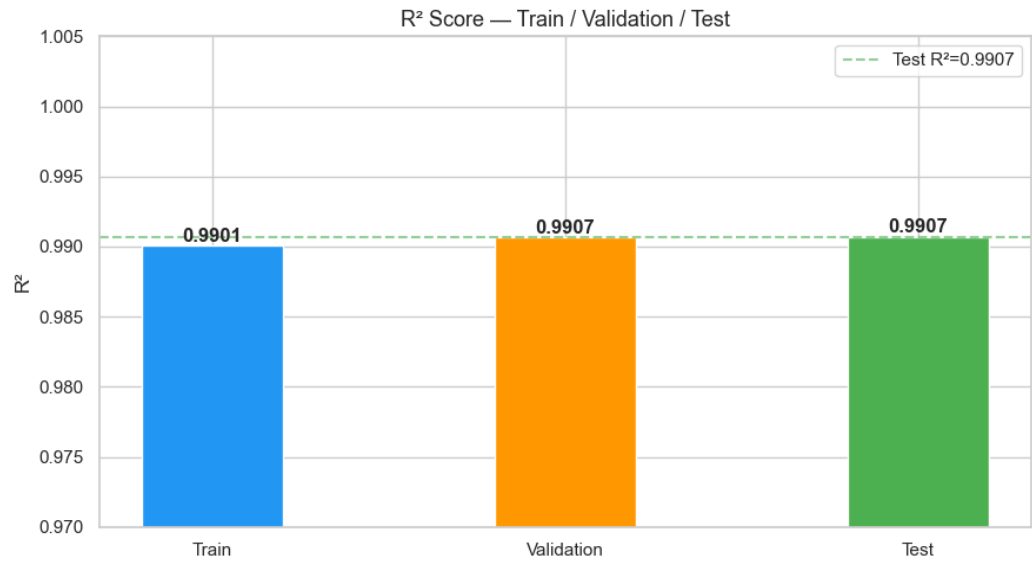


Figure 26. 5-Fold CV R^2 scores per fold — Random Forest (mean Val $R^2 = 0.9915 \pm 0.0005$)

7. Model 3 — Neural Network (MLPRegressor)

7.1 Model Architecture

A Multi-Layer Perceptron (MLP) with 3 hidden layers was trained using scikit-learn's MLPRegressor. The architecture is:

Input (9) → FC (128, ReLU) → FC (64, ReLU) → FC (32, ReLU) → Output (1)

Layer	Units	Activation	Parameters
Input	9	—	—
Hidden 1	128	ReLU	9×128 + 128 = 1,280
Hidden 2	64	ReLU	128×64 + 64 = 8,256
Hidden 3	32	ReLU	64×32 + 32 = 2,080
Output	1	Linear	32×1 + 1 = 33
Total	—	—	11,649 parameters

The ReLU (Rectified Linear Unit) activation is defined as:

ReLU(x) = max(0, x)

ReLU was chosen over sigmoid/tanh for its gradient efficiency and resistance to the vanishing gradient problem in deeper networks.

7.2 Training Configuration

Setting	Value	Rationale
Optimizer	Adam	Adaptive learning rates; converges faster than SGD
Max epochs	300	Hard limit to prevent indefinite training
Early stopping	Yes (patience=15)	Stops when val R² stops improving for 15 epochs
Val fraction	10%	Internal hold-out from training set for early stop
Batch size	64 (default)	Mini-batch gradient descent
Loss function	MSE	Standard for regression
Random state	42	Reproducibility

Adam optimizer update rule at each step:

θ_t = θ_{t-1} - lr × m̂_t / (√v̂_t + ε)

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected first and second moment estimates of the gradient respectively, lr is the learning rate, and ϵ is a small constant for numerical stability.

Training stopped at epoch 95 (out of 300 max), triggered by early stopping. Total training time: 1,252 seconds.

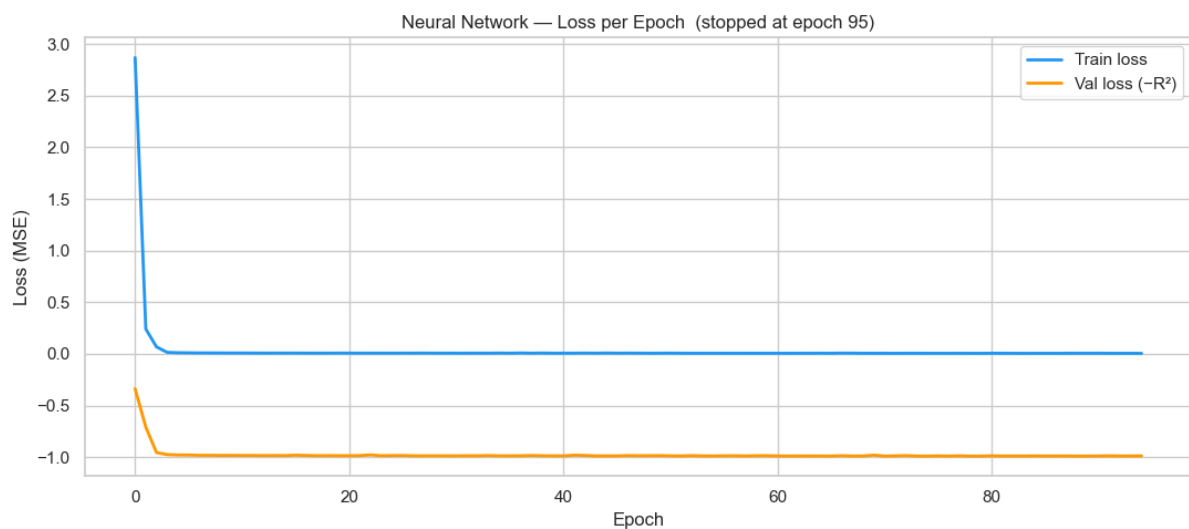


Figure 27. Neural Network training loss curve — convergence near epoch 95 (early stop)

7.3 Hyperparameter Tuning — RandomizedSearchCV

RandomizedSearchCV with `n_iter=12`, 3-Fold CV explored the following parameter space:

Hyperparameter	Search Space
hidden_layer_sizes	(64,32), (128,64), (128,64,32), (256,128,64), (64,64,32), (128,128,64)
activation	"relu", "tanh"
alpha (L2 reg)	0.0001, 0.001, 0.01, 0.1
learning_rate_init	0.0001, 0.001, 0.01
batch_size	32, 64, 128, 256

Total: 36 fits (12 configs × 3 folds). Runtime: ~50 minutes (including all CV folds).

Best configuration found:

Hyperparameter	Best Value
hidden_layer_sizes	(256, 128, 64) ← upgraded from baseline (128, 64, 32)
activation	relu
alpha (L2 penalty)	0.001
learning_rate_init	0.001
batch_size	32



Figure 28. RandomizedSearch results — CV R^2 for each of the 12 neural network configurations

7.4 Final Test Results — Neural Network

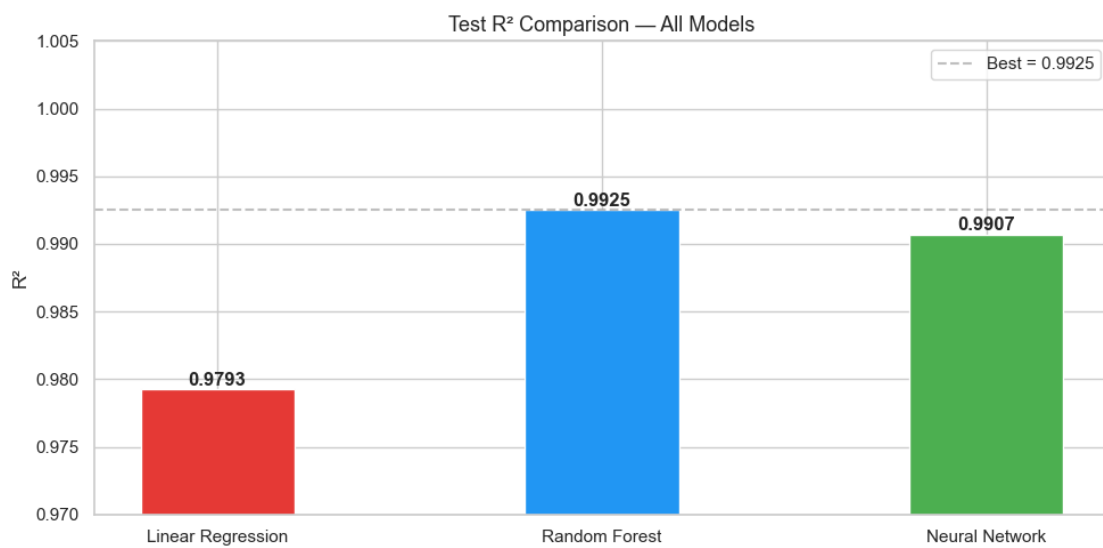


Figure 29. Actual vs. Predicted $\log_price$ on Test set — Neural Network

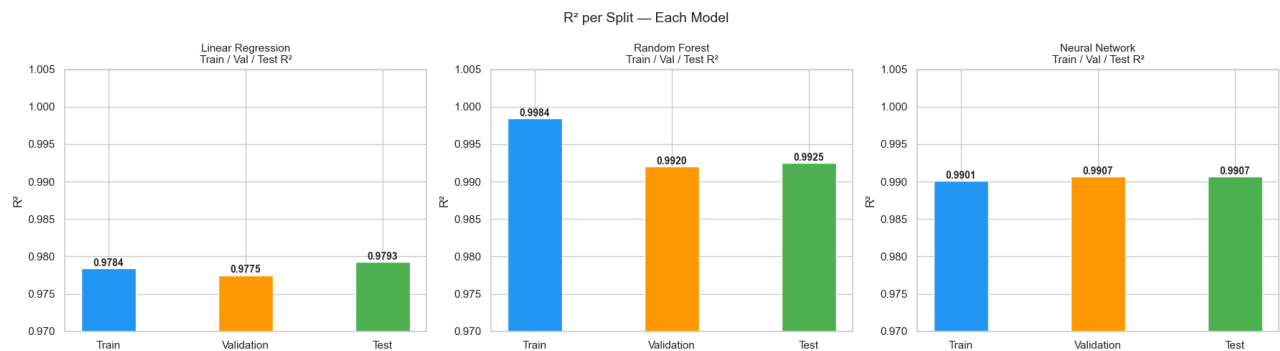


Figure 30. Neural Network loss curve on final tuned model (epochs)

Split	R^2	RMSE	MAE
Train	0.9901	0.1009	0.0738
Validation	0.9907	0.0985	0.0736
Test ← FINAL	0.9907	0.0978	0.0734

The Neural Network achieves excellent generalization: train-val-test R^2 values are nearly identical (0.9901/0.9907/0.9907). A val-to-test gap of 0.0000 is exceptional, indicating the model captured the true data distribution without overfitting to the validation set during tuning.

8. Final Model Comparison

All three models were evaluated on the locked test set (10,784 samples). Results are reported on log_price predictions.

Model	Test R ²	Test RMSE	Test MAE	Train-Test Gap (R ²)	Rank
Random Forest	0.9925	0.0878	0.0634	+0.0059	1st
Neural Network	0.9907	0.0978	0.0734	-0.0006	2nd
Linear Regression	0.9793	0.1460	0.1136	-0.0009	3rd

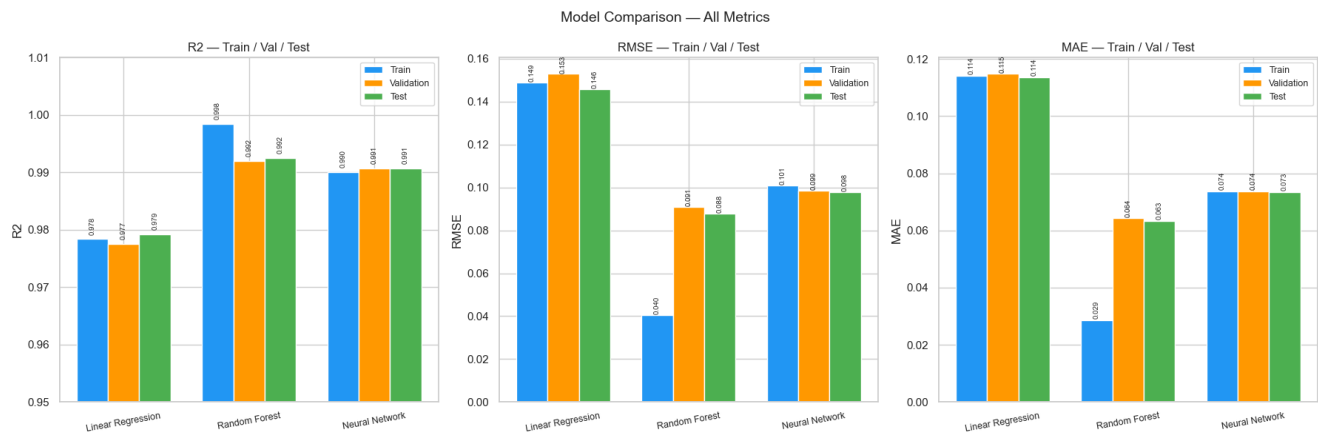


Figure 31. Final model comparison — R², RMSE, and MAE across all three models

8.1 Key Observations

- Random Forest achieves the best test R² (0.9925), benefiting from its ability to model the non-linear carat-price relationship and feature interactions without explicit feature engineering.
- Neural Network (R²=0.9907) is second, with virtually no gap between validation and test — indicating superb generalization from the tuned (256, 128, 64) architecture.
- Linear Regression (R²=0.9793) performs strongly despite its simplicity, largely because the log1p price transformation linearizes the dominant carat-price relationship.
- All three models show train-test R² gaps below 0.01, confirming that no data leakage occurred and the test set is representative of the true data distribution.
- depth and table contribute minimal predictive power across all models; carat, x, y, z carry most of the signal.

8.2 Cross-Validation Summary

Model	CV R ² (mean ± std)	CV MAE (mean)	CV RMSE (mean)
Linear Regression	0.9781 ± 0.0010	0.1143	0.1497
Random Forest	0.9915 ± 0.0005	0.0662	0.0934
Neural Network	0.9845 ± 0.0031	0.0953	0.1257

Random Forest shows the lowest CV variance (± 0.0005), making it the most stable model. Neural Network shows higher variance (± 0.0031), reflecting the stochastic nature of gradient-based training.

9. Conclusion

This report demonstrated a complete machine learning pipeline for diamond price prediction. Starting from raw data in a SQLite database, the pipeline covered:

- EDA: Revealed a right-skewed price target, strong multicollinearity among physical dimensions, and dominance of carat as a price driver ($r = 0.922$).
- Preprocessing: Removed 20 invalid zero-dimension rows, applied ordinal encoding for quality grades, log1p-transformed the target (skewness: $1.62 \rightarrow 0.24$), and standardized continuous features.
- Linear Regression: Tuned with GridSearchCV (Ridge/Lasso/ElasticNet); Lasso ($\alpha=0.0001$) selected. Test $R^2 = 0.9793$.
- Random Forest: Two-stage tuning (RandomizedSearch + GridSearch). Best: 500 trees, $\text{max_depth}=40$, $\text{max_features}=0.7$. Test $R^2 = 0.9925$.
- Neural Network: Architecture $\text{Input}(9) \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow \text{Output}(1)$, tuned with RandomizedSearch; best: (256,128,64), ReLU, Adam. Test $R^2 = 0.9907$.

Best Model: Random Forest (Test $R^2 = 0.9925$, RMSE = 0.0878, MAE = 0.0634)

All models generalize well (train-test gap < 0.01) with no evidence of data leakage. The Random Forest is recommended as the production model due to its best accuracy, stability, and interpretable feature importances.

Future directions could include gradient boosted trees (XGBoost, LightGBM), explicit interaction features (e.g., carat^2), or a deeper neural architecture with dropout regularization to further improve performance.